# CODING_STANDARDS.md

# TeamSync Coding Standards

Version: 1.0

Purpose:

This document establishes coding conventions, architecture rules, naming standards, API standards, and Git workflow guidelines for TeamSync.

All future development phases must follow this document.

This document is considered a source of truth alongside:

- PRD.md
- TRD.md
- APP_FLOW.md
- BACKEND_SCHEMA.md
- API_SPEC.md
- SYSTEM_DESIGN.md
- UI_UX_DESIGN_BRIEF.md
- IMPLEMENTATION_PLAN.md

---

# GENERAL PRINCIPLES

Code should be:

- Readable
- Consistent
- Scalable
- Maintainable
- Production Ready

Avoid:

- Hardcoded values
- Duplicate code
- Business logic in controllers
- Large components
- Large service methods

Prefer:

- Reusable components
- Clear naming
- Separation of concerns
- Type safety
- Validation

---

# GIT CONVENTIONS

Branch Naming

feature/authentication

feature/projects

feature/tasks

feature/comments

feature/notifications

bugfix/login-validation

refactor/user-service

---

Commit Message Convention

docs:

Example

docs: add project documentation

---

feat:

Example

feat: implement jwt authentication

feat: add project management module

---

fix:

Example

fix: resolve login token issue

---

refactor:

Example

refactor: optimize project service

---

style:

Example

style: improve dashboard layout

---

test:

Example

test: add authentication tests

---

chore:

Example

chore: update dependencies

---

# BACKEND STANDARDS

Language

Java 21

Framework

Spring Boot 3

---

Package Structure

com.teamsync

config

security

controller

service

repository

entity

dto

mapper

exception

util

---

Controller Naming

AuthController

UserController

ProjectController

TaskController

CommentController

NotificationController

AnalyticsController

---

Service Naming

AuthService

UserService

ProjectService

TaskService

CommentService

NotificationService

AnalyticsService

---

Repository Naming

UserRepository

ProjectRepository

TaskRepository

CommentRepository

NotificationRepository

---

Entity Naming

User

Project

Task

Comment

Notification

Attachment

ActivityLog

Use singular names only.

---

DTO Naming

Request DTOs

RegisterRequest

LoginRequest

CreateProjectRequest

CreateTaskRequest

---

Response DTOs

AuthResponse

UserResponse

ProjectResponse

TaskResponse

---

Mapper Naming

UserMapper

ProjectMapper

TaskMapper

CommentMapper

---

# DATABASE STANDARDS

Primary Keys

UUID

Example

id UUID

Never use auto increment IDs.

---

Audit Fields

Every table must contain:

created_at

updated_at

---

Boolean Fields

Use

is_active

is_deleted

is_read

Never use:

active

deleted

read

---

# API STANDARDS

Base URL

/api/v1

---

Plural Resource Naming

Correct

/users

/projects

/tasks

/comments

Incorrect

/user

/project

/task

---

HTTP Methods

GET

Retrieve

POST

Create

PUT

Full Update

PATCH

Partial Update

DELETE

Remove

---

# STANDARD SUCCESS RESPONSE

{ "success": true, "message": "Operation completed successfully", "data": {} }

---

# STANDARD ERROR RESPONSE

{ "success": false, "message": "Invalid credentials", "errors": [] }

---

# VALIDATION RULES

Backend validation is mandatory.

Never rely only on frontend validation.

Use:

Spring Validation

@NotBlank

@Email

@Size

@Pattern

@NotNull

---

# SECURITY STANDARDS

Authentication

JWT

Authorization

Role Based Access Control

Roles

ADMIN

LEAD

MEMBER

---

Passwords

Store only BCrypt hashes.

Never store plain text passwords.

---

Secrets

Never hardcode:

Database credentials

JWT secret

Cloudinary credentials

Email credentials

API keys

Use environment variables.

---

# FRONTEND STANDARDS

Language

TypeScript

No JavaScript files.

Use:

.ts

.tsx

---

Folder Structure

src

components

pages

layouts

services

hooks

types

utils

context

theme

assets

---

Component Naming

PascalCase

Correct

ProjectCard.tsx

TaskBoard.tsx

UserAvatar.tsx

Incorrect

projectcard.tsx

taskboard.tsx

---

Page Naming

DashboardPage.tsx

LoginPage.tsx

ProjectsPage.tsx

TaskDetailsPage.tsx

---

Hook Naming

useAuth

useProjects

useTasks

---

Type Naming

User

Project

Task

Comment

CreateProjectPayload

TaskStatus

UserRole

---

# REACT QUERY STANDARDS

Query Keys

["projects"]

["project", projectId]

["tasks"]

["task", taskId]

---

Do not use raw fetch calls.

Use Axios only.

---

# UI STANDARDS

Theme

Dark First

Follow UI_UX_DESIGN_BRIEF.md

---

Buttons

Rounded

Consistent spacing

Large click targets

---

Spacing System

8px

16px

24px

32px

40px

48px

64px

---

Animations

Framer Motion

Maximum Duration

250ms

Avoid:

Bounce

Elastic

Long animations

---

# RESPONSIVENESS

Required Breakpoints

Mobile

Tablet

Desktop

Large Desktop

No desktop-only pages.

---

# FILE STORAGE STANDARDS

Provider

Cloudinary

Store only:

URL

Public ID

in database.

Never store files directly in MySQL.

---

# LOGGING

Use structured logging.

Log:

Authentication Events

Project Creation

Task Creation

Critical Errors

Avoid logging:

Passwords

JWT Tokens

Secrets

---

# TESTING STANDARDS

Future Phases

Unit Tests

Service Tests

Integration Tests

Security Tests

---

# AI AGENT RULES

Before implementing any phase:

1. Read all files in docs/
2. Use docs as source of truth
3. Implement only requested phase
4. Do not modify future phases
5. Do not introduce new architecture patterns
6. Follow naming conventions
7. Follow API response standards
8. Follow folder structure standards
9. Follow UI_UX_DESIGN_BRIEF.md
10. Keep code production ready

---

# DEFINITION OF DONE

A feature is complete only when:

- Backend implemented
- Frontend implemented
- Validation added
- Error handling added
- Responsive UI added
- Loading states added
- Success states added
- Empty states added
- Code follows standards
- Git commit created

Feature implementation without the above is not considered complete.